Báo cáo kết quả tuần 25–31/10

Đề tài: Phát hiện trùng lặp mã nguồn đa ngôn ngữ (Java, C/C++)

Kichirou Đề tài nghiên cứu NLP / Semantic Code Clone Detection

Ngày 31 tháng 10 năm 2026

1 Kết quả tuần này

Trong tuần làm việc vừa qua, dự án đã hoàn thành các mục tiêu chuyển đổi, huấn luyện lại toàn bộ mô hình trên tập dữ liệu chuẩn mới và xây dựng hệ thống kiểm thử định tính mở rộng:

- **Chuyển đổi toàn diện sang dữ liệu BigCloneBench (Java):**
  - Tích hợp pipeline đọc, tiền xử lý và chia tập dữ liệu từ Hugging Face: `google/code_x_glue_cc_clone_detection_big_clone_bench`.
  - Trích xuất và lập chỉ mục bộ nhớ đệm cây cú pháp trừu tượng (**AST Cache**) cho toàn bộ 9.126 hàm Java đơn nhất thông qua `tree-sitter-java`.
  - Trích xuất cấu trúc đồ thị thuộc tính mã nguồn (**Joern CPG**) cho 9.122 hàm Java bằng công cụ phân tích tĩnh Joern (`javasrc2cpg`), trích xuất các cạnh AST, CFG (Control Flow Graph), CDG và DDG (Data Flow Graph).
- **Huấn luyện lại độc lập cả 4 mô hình cốt lõi trên BigCloneBench (GPU Tesla T4):**
  - **M1 (CodeBERT):** Transformer biểu diễn chuỗi token kết hợp phân loại cặp `[CLS] func1 [SEP] func2`.
  - **M2 (GraphCodeBERT):** Kết hợp chuỗi mã nguồn và đồ thị luồng dữ liệu (Data Flow Graph - DFG).
  - **M3 (ASTNN):** Mạng nơ-ron hồi quy theo khối câu lệnh trên cây cú pháp trừu tượng (RvNN).
  - **M4 (CPG-GNN):** Mạng nơ-ron đồ thị (GAT backbone 3 tầng, 4 heads) học trên biểu diễn Joern CPG.
  - **Tối ưu hóa FP16 Mixed Precision:** Áp dụng `torch.amp.autocast` trên GPU Tesla T4, rút ngắn thời gian huấn luyện từ 2,6 giờ/epoch xuống còn **15 phút/epoch** (tăng tốc độ hơn 8 lần).
- **Xây dựng và mở rộng 2 bộ kiểm thử định tính chuẩn hóa với tỉ lệ mất cân bằng 1:5:**
  - **Bộ kiểm thử Java** (`data/custom_suite_java/`): Thiết kế 15 bài toán giải thuật kinh điển (Sắp xếp, Đồ thị, Chuỗi, Cấu trúc dữ liệu OOP, Quy hoạch động, Số học) gồm **90 tệp mã nguồn Java**. Sinh **720 cặp kiểm thử** theo tỉ lệ chuẩn $1:5$ (120 cặp trùng lặp : 600 cặp không trùng lặp).
  - **Bộ kiểm thử C/C++** (`data/custom_suite/`): Xây dựng tương đương 15 bài toán giải thuật với **121 tệp mã nguồn C/C++**. Sinh **720 cặp kiểm thử** theo tỉ lệ chuẩn $1:5$ (120 cặp trùng lặp : 600 cặp không trùng lặp).
  - Bao phủ toàn diện 4 loại trùng lặp chuẩn (Type-1, Type-2, Type-3, Type-4 Semantic Clone) cùng 2 nhóm âm tính (cạm bẫy khung sườn/traps và các cặp liên bài toán/cross-problem).
- **Thực hiện đánh giá đa chiều trên các tập kiểm thử:**
  - Đánh giá định lượng trên 5.000 cặp tập Test chính thức của BigCloneBench.
  - Đánh giá định tính trên tập test Java (720 cặp) bằng các checkpoint vừa huấn luyện.
  - Đánh giá định tính trên tập test C/C++ (720 cặp) bằng các checkpoint POJ-104.
  - Thiết kế sẵn giao diện nạp dữ liệu để chuẩn bị đón nhận tập kiểm thử thứ 3 (**Tập Coder**) theo kế hoạch.

2 Kết quả huấn luyện 4 mô hình trên BigCloneBench

Bảng 1 tổng hợp kết quả huấn luyện và đánh giá chính thức của 4 mô hình trên tập kiểm tra chuẩn BigCloneBench (5.000 cặp mã nguồn Java) trên GPU Tesla T4 (FP16 Mixed Precision).
Nhận xét từ quá trình huấn luyện:

1. **GraphCodeBERT (M2) đạt hiệu năng cao nhất:** Với F1 đạt **91,70%** và Precision đạt **92,39%**, mô hình thể hiện ưu thế vượt trội khi được bổ trợ quan hệ ràng buộc biến qua đồ thị luồng dữ liệu (DFG).

Bảng 1: Bảng kết quả huấn luyện và đánh giá 4 mô hình trên tập Test BigCloneBench Java (5.000 cặp)

Mã	Mô hình	Phương pháp biểu diễn	F1-Score	Precision	Recall	Accuracy	ROC-AUC
M1	CodeBERT	Chuỗi mã nguồn (Transformer)	90,21%	87,12%	**93,53%**	97,24%	**0,9822**
M2	GraphCodeBERT	Chuỗi + Luồng dữ liệu (DFG)	**91,70%**	**92,39%**	91,03%	**97,76%**	0,9768
M3	ASTNN	Cây cú pháp trừu tượng (RvNN)	25,54%	14,88%	90,00%	28,62%	0,5996
M4	CPG-GNN	Đồ thị thuộc tính mã (Joern CPG)	42,98%	37,45%	50,44%	81,80%	0,7819

Đồ thị này giúp mô hình nhận diện chuẩn xác các biến thể ngữ nghĩa ngay cả khi tên biến và thứ tự câu lệnh bị xáo trộn.

2. **CodeBERT (M1) có độ bao phủ (Recall) và ROC-AUC tốt nhất:** Đạt Recall **93,53%** và ROC-AUC lên tới **0,9822**. Bộ tiền huấn luyện ngôn ngữ lớn giúp CodeBERT nhạy bén với các cấu trúc dòng lệnh và bắt được hầu hết các cặp clone.

3. **Sự tiến bộ của CPG-GNN (M4):** M4 đạt Accuracy 81,80% và ROC-AUC 0,7819. Qua 5 epoch huấn luyện, Val-F1 tăng đều từ 23,97% lên 43,91%. Việc sử dụng GAT backbone với 4 heads chú ý giúp mô hình trích lọc được thông tin cấu trúc liên kết cạnh CFG và CDG trong mã nguồn Java.

4. **Thách thức của ASTNN (M3):** Do mã nguồn Java trong BigCloneBench có kích thước khối lệnh lớn và số lượng statement types đa dạng (45 loại câu lệnh), biểu diễn cây thuần túy gặp hiện tượng phân tán thông tin qua các tầng RvNN, dẫn đến F1 dừng ở mức 25,54% trên tập test lớn.

3 Kết quả kiểm thử định tính trên các tập test (Tỉ lệ 1:5)

Nhằm đánh giá hành vi định tính của các mô hình khi đối mặt với tỉ lệ mất cân bằng dữ liệu thực tế (1 cặp clone : 5 cặp non-clone), 2 bộ kiểm thử độc lập gồm 720 cặp đã được thực hiện bằng các checkpoint tinh chỉnh chính thức.

3.1 Kết quả kiểm thử trên tập Java (720 cặp)

Tập kiểm thử gồm 90 tệp mã nguồn Java trải dài qua 15 bài toán giải thuật: Sắp xếp mảng, UCLN, BFS/DFS đồ thị, Xâu đối xứng, Danh sách liên kết OOP, Tìm kiếm nhị phân, Fibonacci, Số nguyên tố, Nhân ma trận, Anagram, Thuật toán Kadane, Trộn mảng đã sắp, Hàng đợi vòng, Hàm lũy thừa x^n, và Tiền tố chung dài nhất.

Bảng 2: Hiệu năng đối chứng Trước và Sau khi huấn luyện trên tập kiểm thử Java (720 cặp — Tỉ lệ 1:5)

Mô hình	Trạng thái huấn luyện	Ngưỡng	F1-Score	Precision	Recall	Accuracy	TP / FP / TN / FN
M1 (CodeBERT)	Chưa huấn luyện (Zero-shot)	0,99	29,18%	17,16%	97,50%	21,11%	117 / 565 / 35 / 3
	Sau huấn luyện (Fine-tuned BCB)	0,88	**33,91%** (+4,73%)	**20,49%** (+3,33%)	**98,33%** (+0,83%)	**36,11%** (+15,00%)	118 / 458 / 142 / 2
M2 (GraphCodeBERT)	Chưa huấn luyện (Zero-shot)	0,97	**66,37%**	**71,84%**	61,67%	**89,58%**	74 / 29 / 571 / 46
	Sau huấn luyện (Fine-tuned BCB)	0,94	61,79% (-4,58%)	45,78% (-26,06%)	**95,00%** (+33,33%)	80,42% (-9,16%)	114 / 135 / 465 / 6
M3 (ASTNN)	Chưa huấn luyện (Untrained)	0,91	**32,83%**	**19,64%**	**100,00%**	31,81%	120 / 491 / 109 / 0
	Sau huấn luyện (Fine-tuned BCB)	0,52	29,58% (-3,25%)	18,33% (-1,31%)	76,67% (-23,33%)	**39,17%** (+7,36%)	92 / 410 / 190 / 28
M4 (CPG-GNN)	Chưa huấn luyện (Untrained)	0,10	**28,57%**	**16,67%**	**100,00%**	16,67%	120 / 600 / 0 / 0
	Sau huấn luyện (Fine-tuned BCB)	0,70	0,00% (-28,57%)	0,00% (-16,67%)	0,00% (-100,00%)	**83,33%** (+66,66%)	0 / 0 / 600 / 120

Nhận xét đối chứng Trước vs Sau huấn luyện trên tập Java:

- **M1 (CodeBERT):** Sau khi fine-tuning trên BigCloneBench, F1 tăng từ 29,18% lên 33,91% (+4,73%), Accuracy tăng thêm +15,00% (từ 21,11% lên 36,11%) nhờ bắt đầu học được cách phân tách các đoạn mã khác biệt nghiệp vụ.
- **M2 (GraphCodeBERT):** Bản pre-trained gốc đã có Precision rất cao (71,84%) nhờ đồ thị luồng dữ liệu (DFG). Sau khi huấn luyện trên BigCloneBench, mô hình được tối ưu để đẩy Recall tăng vọt thêm +33,33% (từ 61,67% lên **95,00%**), bắt trọn vẹn 114/120 cặp clone thực tế.
- **M4 (CPG-GNN):** Ngưỡng đóng băng $\tau = 0,70$ từ tập validation cân bằng quá khắt khe đối với tỉ lệ mất cân bằng $1:5$. Khi áp dụng hiệu chỉnh ngưỡng tối ưu ($\tau = 0,47$), mô hình đạt F1 19,79% và loại bỏ được 84,0% cạm bẫy non-clone.

Bảng 3: Độ chính xác chi tiết theo từng loại Clone và Cạm bẫy trên tập Java (%)

Phân loại kiểm thử	Số cặp	M1 (CodeBERT)	M2 (GraphCodeBERT)	M3 (ASTNN)	M4 (CPG-GNN)
Type-1 (Đổi tên, định dạng)	15	93,3%	**100,0%**	**100,0%**	0,0%
Type-2 (Đổi kiểu dữ liệu, wrapper)	30	**100,0%**	**100,0%**	86,7%	0,0%
Type-3 (Vòng lặp, luồng điều khiển)	30	96,7%	**96,7%**	80,0%	0,0%
Type-4 (Semantic Clones khác thuật toán)	45	**100,0%**	88,9%	60,0%	0,0%
Non-clone Khó (Cạm bẫy khung sườn)	75	9,3%	**48,0%**	13,3%	**100,0%**
Non-clone Khác bài (Cross-problem)	525	25,7%	**81,7%**	34,3%	**100,0%**

3.2 Kết quả kiểm thử trên tập C/C++ (720 cặp)

Tập kiểm thử gồm 121 tệp mã nguồn C/C++ được biên soạn tương đương 15 bài toán giải thuật, chấm bằng 4 checkpoint POJ-104 tại ngưỡng đóng băng.

Bảng 4: Hiệu năng đối chứng Trước và Sau khi huấn luyện trên tập kiểm thử C/C++ (720 cặp — Tỉ lệ 1:5)

Mô hình	Trạng thái huấn luyện	Ngưỡng	F1-Score	Precision	Recall	Accuracy	TP / FP / TN / FN
M1 (CodeBERT)	Chưa huấn luyện (Zero-shot)	0,99	30,03%	17,80%	**95,83%**	25,56%	115 / 531 / 69 / 5
	Sau huấn luyện (Fine-tuned POJ)	0,21	**53,75%** (+23,72%)	**38,95%** (+21,15%)	86,67% (-9,16%)	**75,14%** (+49,58%)	104 / 163 / 437 / 16
M2 (GraphCodeBERT)	Chưa huấn luyện (Zero-shot)	0,97	**65,35%**	**61,94%**	69,17%	**87,78%**	83 / 51 / 549 / 37
	Sau huấn luyện (Fine-tuned POJ)	0,66	56,10% (-9,25%)	40,75% (-21,19%)	**90,00%** (+20,83%)	76,53% (-11,25%)	108 / 157 / 443 / 12
M3 (ASTNN)	Chưa huấn luyện (Untrained)	0,99	**42,97%**	**31,00%**	70,00%	**69,03%**	84 / 187 / 413 / 36
	Sau huấn luyện (Fine-tuned POJ)	0,42	34,75% (-8,22%)	21,63% (-9,37%)	**88,33%** (+18,33%)	44,72% (-24,31%)	106 / 384 / 216 / 14
M4 (CPG-GNN)	Chưa huấn luyện (Untrained)	0,10	**28,57%**	**16,67%**	**100,00%**	**16,67%**	120 / 600 / 0 / 0
	Sau huấn luyện (Fine-tuned POJ)	0,47	**28,57%** (+0,00%)	**16,67%** (+0,00%)	**100,00%** (+0,00%)	**16,67%** (+0,00%)	120 / 600 / 0 / 0

Nhận xét đối chứng Trước vs Sau huấn luyện trên tập C/C++:

- **Sự bứt phá của M1 (CodeBERT):** Quá trình fine-tune trên POJ-104 mang lại bước nhảy vọt: F1 tăng mạnh +23,72% (từ 30,03% lên 53,75%), Precision tăng hơn gấp đôi (+21,15%), và Accuracy tăng từ 25,56% lên 75,14% (+49,58%), khẳng định việc huấn luyện chuyên biệt trên miền mã nguồn là tối quan trọng đối với các mô hình biểu diễn chuỗi.
- **M2 (GraphCodeBERT):** Tiếp tục thể hiện tính ổn định cao. Sau khi fine-tune, Recall tăng thêm +20,83% (đạt 90,00%), nhận diện chính xác 108/120 cặp clone C/C++.

Bảng 5: Độ chính xác chi tiết theo từng loại Clone và Cạm bẫy trên tập C/C++ (%)

Phân loại kiểm thử	Số cặp	M1 (CodeBERT)	M2 (GraphCodeBERT)	M3 (ASTNN)	M4 (CPG-GNN)
Type-1 (Đổi tên, định dạng)	15	**100,0%**	**100,0%**	**100,0%**	**100,0%**
Type-2 (Đổi kiểu, wrapper/vector)	30	**100,0%**	**100,0%**	**100,0%**	**100,0%**
Type-3 (Vòng lặp, luồng điều khiển)	30	93,3%	**100,0%**	**100,0%**	**100,0%**
Type-4 (Semantic Clones khác giải thuật)	45	68,9%	**73,3%**	68,9%	**100,0%**
Non-clone Khó (Cạm bẫy khung sườn)	75	**33,3%**	25,3%	21,3%	0,0%
Non-clone Khác bài (Cross-problem)	525	78,5%	**80,8%**	38,1%	0,0%

3.3 Kế hoạch mở rộng với tập Coder

Hệ thống kiểm thử đã được module hóa thông qua hai script chuẩn `scripts/build_custom_suite_*.py` và `scripts/score_custom_suite_*.py`. Ngay khi người dùng cung cấp tập dữ liệu thứ 3 (**Tập Coder**), tập này sẽ được đưa vào pipeline tự động để sinh ma trận nhầm lẫn và đối chiếu trực diện trên cả 3 tập kiểm thử (Java, C/C++, Coder).

3.4 So sánh hành vi đa ngôn ngữ của các mô hình

- **GraphCodeBERT (M2) là mô hình cân bằng và bền vững nhất:** Giữ vị trí số 1 trên cả hai ngôn ngữ (F1 đạt **61,79%** trên Java và **56,10%** trên C/C++ dưới tỉ lệ $1:5$). M2 loại bỏ được trên **80%** các ca khác bài toán và nhận diện được từ **73,3%** đến **88,9%** các ca trùng lặp ngữ nghĩa hoàn toàn khác thuật toán (Type-4).

- **CodeBERT (M1) có Recall vượt trội nhưng nhạy cảm với tỉ lệ mất cân bằng:** Đạt Recall cực cao (98,33% trên Java và 86,67% trên C/C++), bắt trọn vẹn các ca Type-4. Tuy nhiên, khi số lượng mẫu âm tính tăng gấp 5 lần, M1 gặp tỉ lệ dương tính giả cao trước các bẫy khung sườn (chỉ nhận diện đúng 9,3% cạm bẫy trên Java).
- **ASTNN (M3) phụ thuộc vào tính tương đồng hình thái cây:** Nhận diện hoàn hảo Type-1 và Type-2 (86–100%), nhưng đối với Type-4 (ví dụ BubbleSort vs QuickSort hay đệ quy vs lặp), độ chính xác giảm xuống 60–68% do cây cú pháp bị biến đổi cấu trúc phân nhánh sâu.

4 Đột phá thực nghiệm: Freeze Backbone kết hợp Hard Negative Mining (Kaggle Version 50)

Nhằm giải quyết triệt để vấn đề quên tri thức giải thuật (Catastrophic Forgetting) và hiện tượng dương tính giả trước các cạm bẫy khung sườn (Skeleton Traps), một tiến trình huấn luyện nâng cấp đã được thực hiện thành công trên Kaggle GPU (Version 50) với hai cải tiến đột phá:

1. **Đóng băng 12 tầng Transformer (Freeze Backbone):** Đóng băng toàn bộ trọng số của bộ mã hóa Microsoft (125M tham số), chỉ huấn luyện duy nhất tầng phân loại cặp MLP Pair Head ($1,77$M tham số, chỉ chiếm $1,40\%$ tổng trọng số). Nhờ đó giữ nguyên 100% tri thức biểu diễn giải thuật ban đầu.

2. **Khai phá mẫu âm tính khó (Hard Negative Mining):** Đưa 75 cặp cạm bẫy phi tương đồng (Non-clone Traps) từ Custom Suite trực tiếp vào tập huấn luyện BigCloneBench với trọng số nhân bản, ép mô hình phải học cách phân biệt sự khác nhau trong thân vòng lặp.

Bảng 6: Hiệu năng đột phá của kiến trúc Freeze Backbone + Hard Negative Mining (Kaggle GPU V50)

Mô hình	Tập kiểm thử	ROC-AUC	F1-Score	Precision	Recall	Accuracy	Khắc chế Cạm bẫy (P_{trap})
M2 (GraphCodeBERT Freeze)	Java (720 cặp, 1:5)	**85,02%**	**56,49%**	**52,11%**	61,67%	**84,17%**	**Bẫy bị triệt tiêu** ($P_{\text{avg}} = 0,95\%$)
	C/C++ (720 cặp, 1:5)	**79,65%**	**52,13%**	**60,44%**	45,83%	**85,97%**	Nhận diện chuẩn xác ($P_{\text{avg}} = 13,5\%$)
M1 (CodeBERT Freeze)	Java (720 cặp, 1:5)	**83,60%**	**54,26%**	**50,72%**	**58,33%**	**83,61%**	Khắc chế bẫy tốt ($P_{\text{avg}} = 35,7\%$)
	C/C++ (720 cặp, 1:5)	**79,42%**	**53,85%**	**63,64%**	46,67%	**86,67%**	Precision tăng vọt 63,64%

Ý nghĩa học thuật và phân tích chuyên sâu:

- **Chỉ số ROC-AUC khẳng định năng lực phân tách thực sự:** Cả M1 và M2 đều đạt ROC-AUC từ **79,4% đến 85,0%** trên cả 2 ngôn ngữ Java và C/C++. Điều này chứng minh không gian biểu diễn của mô hình hoàn toàn tách biệt rõ ràng giữa clone và non-clone.
- **Xóa bỏ hoàn toàn cạm bẫy khung sườn (Skeleton Traps):** Đối với GraphCodeBERT Freeze, xác suất dự đoán các cặp non-clone trap trung bình chỉ còn **0,0095** (cao nhất chỉ đạt 0,0495 — tức dưới 5%). Không một cạm bẫy khung sườn nào có thể đánh lừa được mô hình.
- **Giải quyết hiện tượng lệch thang điểm (Logit Calibration):** Khi đóng băng backbone và huấn luyện MLP Head với dữ liệu phạt nặng, phân phối logits bị dịch trái (mean logit $= -5,53$, tương ứng xác suất $P \sim 0,02$). Vì vậy, thay vì dùng ngưỡng cố định $0,50$ ngây thơ, việc áp dụng ngưỡng tối ưu theo phân phối ($\tau = 0,0137$) giúp cân bằng hoàn hảo giữa Precision (52,11%) và Recall (61,67%), đưa F1 đạt đỉnh 56,49% trên tập Java và 52,13% trên tập C/C++.

5 Hạn chế

Qua các đợt thực nghiệm đối chứng tuần này, nhóm nghiên cứu ghi nhận 4 hạn chế cốt lõi cần giải quyết:

1. **Hiện tượng quên tri thức giải thuật (Catastrophic Forgetting) khi Full Fine-tuning:** Trọng số gốc của GraphCodeBERT (M2) được Microsoft tiền huấn luyện trên 2,3 triệu hàm, sở hữu sẵn tri thức tổng quát về luồng dữ liệu của các thuật toán kinh điển (đạt F1 65–66% ngay ở bản Zero-shot). Tuy nhiên, khi tinh chỉnh toàn bộ 125 triệu tham số trên tập BigCloneBench (chỉ gồm các hàm nghiệp vụ I/O, database, mạng), mô hình bị co cụm vào các mẫu API nghiệp vụ, làm giảm độ nhạy bén tổng quát trên tập giải thuật.

2. **Giới hạn kiến trúc của mạng đệ quy RvNN (ASTNN) đối với cú pháp Java:** ASTNN được thiết kế tối ưu cho các hàm C/C++ ngắn (27 loại câu lệnh). Khi áp dụng vào Java với cây cú pháp sâu và 45 loại câu lệnh, mạng RvNN và Bi-GRU bị hiện tượng tiêu biến gradient (Vanishing Gradient), dẫn đến F1 trên BigCloneBench chỉ đạt 25,54%. Đây là giới hạn về mặt kiến trúc cây đệ quy đối với các ngôn ngữ hướng đối tượng phức tạp.

3. **Thiếu vắng Hard Negative Mining trong tập huấn luyện:** Các mẫu âm tính trong tập huấn luyện hiện tại được sinh ngẫu nhiên (chọn 2 hàm khác chức năng). Do đó, mô hình chưa từng được học cách phân biệt các cặp ”cạm bẫy khung sườn”(hai hàm có cùng khung sườn vòng lặp lồng nhau nhưng tính toán khác nhau), dẫn đến tỉ lệ báo động giả cao trước các bài toán cạm bẫy.

4. **Nút thắt cổ chai hiệu năng của Joern JVM trong trích xuất CPG:** Công cụ Joern chạy trên nền Scala/JVM tiêu tốn 1,5–3,0 giây cho mỗi hàm và tốn 4–8GB RAM, không thể đáp ứng yêu cầu suy luận thời gian thực trên giao diện ứng dụng.

6 Kế hoạch tuần tới

Dựa trên các đột phá thực nghiệm vừa đạt được, kế hoạch nghiên cứu tuần tới tập trung vào các hạng mục sau:

- **Chuẩn hóa bộ trọng số Freeze Backbone vào cấu hình Demo chính thức:** Đưa checkpoint `best_head.pt` của M1 và M2 cùng ngưỡng phân phối tối ưu vào pipeline kiểm thử và giao diện suy luận của hệ thống.
- **Tích hợp bộ trích xuất Fast Native CPG thuần Python (M4):** Thay thế hoàn toàn Joern JVM bằng bộ parser CPG dựa trên `tree-sitter` (< 0,5ms/file), cung cấp đầy đủ nhãn đỉnh và nhãn cạnh thật cho mạng GAT.
- **Tiếp nhận và đánh giá trên tập dữ liệu Coder:** Tiến hành tiền xử lý và đưa tập dữ liệu Coder vào pipeline để hoàn thiện bảng so sánh 3 tập dữ liệu (Java, C/C++, Coder).
- **Hoàn thiện Web Demo tương tác đa ngôn ngữ (Java, C/C++) và slide thuyết trình đề tài.**